From Semantics to Decisions: A Federated Vision-Language Decision Model for UAV Power-Line Inspection

Mahdi Khashan

[a][Affiliation to be added],

Abstract

A recent agenda for edge-native semantic UAV inspection shows that a lightweight vision-language model (VLM) runs on a Jetson-class drone computer at 2.8 s per image, and leaves speed, measured reliability, training data and fleet collaboration open. We study a vision-language decision model, laya-vision (201M parameters), which answers typed inspection questions in one forward pass with calibrated probabilities, not generated text. On 35,415 power-line component images (ElecVQA-Bench, InsPLAD), the published checkpoint fails zero-shot (condition macro F1 0.180). Fine-tuning on 4,807 images with labels from the dataset folders raises it to 0.703, with component accuracy 0.924 and defect AUROC 0.965, at 74 ms per image for four questions on an RTX 2080 Ti. As a trigger, the defect probability passes 15.3% of images on and finds 88.5% of defects. With FedAvg over simulated tower-wise drones, accuracy falls as the data is split further (macro F1 0.624, 0.541, 0.474 for 3, 6, 18 drones) but stays above drones that train alone (0.371), and a start fine-tuned on separate server towers brings the fleet to 0.636. From the published checkpoint, small updates (head, LoRA, last four layers; 17–45 MB) learn little in ten rounds (macro F1 0.24–0.28), more rounds help (0.49 after 30), int8 updates cost nothing measurable and FedProx changes nothing. With 100–200 natural labels per drone, fleet FedAvg from the server model raises defect AUROC from 0.903 to 0.93, below pooling the same labels (0.958). Drones that each see different components learn their own defect type alone, which small-update FedAvg loses for every split.

Keywords: semantic edge intelligence, vision-language model, decision model, federated learning, UAV inspection, power lines

1. Introduction

Farahani et al. [1] describe how inspection pipelines move from cloud-centric image analysis, through onboard detectors such as YOLO, to edge-native semantic inspection, in which a lightweight vision-language model (VLM) on the drone interprets what the detector found. Their feasibility study on an NVIDIA Jetson Orin shows that this is possible but slow: a fine-tuned SmolVLM-256M [4] needs about 2.8 s per image, against 0.14 s for YOLO26M, and continuous captioning of a 10 s, 30 fps clip takes about 14 min [1]. The same group's EVLM system [2] reports 1.4–6.4 s per 4 s video segment for a LoRA-tuned SmolVLM on power-line inspection, and a companion study profiles cloud, onboard and split deployment of such models [3].

The agenda in [1] lists the system challenges that remain (its Table 1) and the directions that follow. Five of them are the starting point of this paper:

1. Low-latency multimodal reasoning: semantic inference must fit the flight.
2. Hierarchical semantic processing: an inexpensive stage should decide when the expensive VLM runs.
3. Multimodal training data: image–text inspection data is scarce, and the study relies on captions generated by GPT-4o.
4. Trustworthy autonomy: outputs need confidence estimates that a mission planner can act on.
5. Collaborative edge intelligence: many UAVs should improve one model while models run locally on each drone.

The feasibility study shows semantic quality with three qualitative examples, so the gain from fine-tuning is not measured.

Many inspection questions do not need free text. "Does this clamp have a defect?" and "Which condition does this insulator have?" are choices among known options. Decision models answer such typed questions in one forward pass and return a probability for each option. Laya [5] introduced the design for text, and laya-vision [6] replaces its text encoder with a truncated SmolVLM backbone, so that the input can be an image. A decision model is therefore a natural middle stage between a detector and a describing VLM: it keeps the vision-language backbone, but gives labels and probabilities at a cost close to one image encoding.

The images that would improve such a model are spread across the fleet that collected them. Different operators

Email address: mahdikhashan1@gmail.com (Mahdi Khashan)

may own the drones, and imagery of critical infrastructure is sensitive. Federated learning [9] trains one model across clients without moving their data, but inspection data differs strongly between drones, because each drone flies its own towers.

This paper asks four questions, each tied to an open problem of [1]:

RQ1 How well does a vision-language decision model classify power-line components and their condition, zero-shot and after fine-tuning on labels that need no caption generation? (Challenges 3 and the evaluation gap.)

RQ2 What is its latency, and can its calibrated output act as the trigger of a hierarchical pipeline? (Challenges 1, 2 and 4.)

RQ3 How close does federated fine-tuning across tower-wise drones come to central fine-tuning? (Challenge 5.)

RQ4 Does a model fine-tuned centrally on separate data help the federated fleet? (Challenge 5.)

RQ5 Which part of the model must each drone send, and how do few labels per drone and drones with different image types change the result? (Challenge 5 and communication.)

Our contributions are a gap map from the challenges of [1] to measured evidence (Table 1); a quantitative evaluation of a vision-language decision model on 35,415 InsPLAD images from zero-shot to fine-tuned, against a fine-tuned ResNet-50 and a LoRA-tuned SmolVLM; a latency and trigger analysis; and a federated study with towers as drones that covers the number of drones, a fine-tuned start, the size and format of the update, FedProx, few labels per drone and drones with different image types.

2. Gap analysis and related work

Table 1 maps each open problem to the evidence in this paper. We mark a problem as addressed only when we have a measurement, as partly addressed when the evidence is an estimate or a simulation, and as open otherwise.

Edge VLMs for UAV inspection. EVLM [2] filters key frames and runs a LoRA-tuned SmolVLM-256M on a Jetson for power-line inspection. It reduces the uplink of a 4 s segment from 485 kB to 25 kB at a cost of 1.4–6.4 s of generation per segment. The cloud, edge or split study [3] profiles where such a model should run. INAR-VL [7] routes vision-language requests between edge and cloud with accuracy-aware policies. All of these generate text; we study a model whose outputs are labels and probabilities.

Decision models. Laya [5] scores typed questions (choice, yes/no, score) in one pass with a reinforcement-style objective and a temperature for each question type [8]. Laya-vision [6] adds image input. We use its published checkpoint and its fine-tuning recipe.

Federated learning. FedAvg [9] averages client models weighted by data size. Non-IID client data degrades FedAvg [10], and FedProx [11] adds a proximal term for heterogeneous clients. Federated learning has been applied to the routing of language models [13] and to the uncertainty thresholds that decide when an edge model escalates to the cloud [14]. A recent study combines escalation of uncertain detections with federated adapter updates for guardrail inspection [15]. To our knowledge, federated fine-tuning of a vision-language decision model on drone inspection data has not been reported.

3. Method

3.1. Model and questions

Laya-vision [6] (checkpoint thaitea/laya-vision, revision f2fe3c1, 201M parameters) uses SmolVLM-256M cut to 20 of its 30 language layers, followed by a decision head. For an input and a typed question, the head returns one logit per option. A temperature per question type converts the logits to probabilities. The input in all our experiments is the image only. One predict call per image asks four questions:

- component (choice, 11 options): which power-line component is in the image.
- defect (yes/no): whether the component has a defect.
- condition (choice, 6 options): good, rust, corrosion, missing cap, bird nest, or other defect (torn part or peeling paint).
- judgeable (yes/no): whether the image is clear enough to judge. This question has no ground truth.

The image is encoded once and reused for the four questions. The model can only answer with one of the given options, so it cannot produce a description that contradicts the option set. It also cannot describe a defect outside that set.

3.2. Data and ground truth

ElecVQA-Bench [16] builds visual questions on the InsPLAD component crops [17]. Power-line inspection is one of the four domains of [1], and its anomaly types (degradation, missing component, corrosion) are the classes of this data. We use the tower14 split: 18 training, 10 validation and 14 test towers, with 39,228 items on 35,415 unique images. The towers in different splits share no images. Each image carries its component folder and its class folder. We take the component (17 folders, with the 7 shackle types merged into one, giving 11 classes) and the condition (6 classes, with torn-up and peeling-paint merged) from these folders. A defect is any condition other than good. The folder labels agree with every ElecVQA-Bench answer. The test towers hold 6,092 images, of which 601 (9.9%) show a defect. No caption is generated at any step, so the training labels need no teacher model and no manual text annotation.

Table 1: Open problems of the edge-native semantic inspection agenda [1] and the evidence in this paper. "Est." marks a number derived from a hardware factor, not measured on a Jetson.

Open problem in [1]	Evidence in this paper	Status
Low-latency reasoning (2.8 s per image for SmolVLM-256M)	74 ms per image for four questions on an RTX 2080 Ti; about 0.67 s on a Jetson Orin NX (est.) (Sec. 4.3)	Partly: no Jetson run
Hierarchical processing	Calibrated defect probability as trigger: 15.3% of images passed on, 88.5% defect recall (Sec. 4.4)	Partly: offline
Multimodal training data (GPT-4o captions)	Typed questions with labels from the dataset folders; no generated captions (Sec. 3.2)	Closed-set questions
Measured semantic quality	Macro F1, AUROC and ECE with bootstrap intervals on 6,092 held-out images (Secs. 4.1–4.2)	Addressed
Trustworthy autonomy	Calibrated probabilities (ECE 0.058), risk-based threshold, fixed option set; the self-assessment answer fails (Sec. 4.5)	Partly
Collaborative edge intelligence	FedAvg over 3–18 tower-wise drones, server-fine-tuned start, 50–200 natural labels per drone, drones with different components (Secs. 4.6–4.10)	Partly: simulated
Communication of updates	Five update sizes (16.9–229.5 MB) trained; int8 free; small updates need more rounds (Sec. 4.8)	Partly: calculated
Cross-structure generalization	14 unseen test towers, one asset type	Partly
Energy, mission-aware scheduling, multimodal sensing, digital twins	Not studied	Open

3.3. Fine-tuning

We follow the fine-tuning guide of laya-vision [6]. Each image gives three labelled questions (component, defect, condition). Training uses the RLCD objective of Laya with soft cross-entropy, AdamW, batch 8, learning rates 10^{-4} for the head and 2×10^{-5} for the backbone, 25 warm-up steps and 2.5 passes over the training questions. Temperatures are refit on held-out training images after training [8]. The RTX 2080 Ti has no native bfloat16, so all training runs in fp32. We train three models:

- Round 1: up to 400 images per condition (1,893 images, 79% defects; 1,793 for training), training the head and the last 4 language layers (22.6M parameters).
- Round 2, last 4 layers: all 2,249 defect images of the training towers plus up to 300 good images per component (5,107 images, 44% defects; 4,807 for training, 300 for temperatures).
- Round 2, full: the same data, training all parameters except the vision encoder (114.7M parameters).

3.4. Federated fine-tuning

The simulated drones use the same 5,107 images as round 2, divided by tower. For K drones, towers are assigned to drones so that the image counts are balanced. With $K = 18$ each drone holds one tower, from 1 to 1,103 images, and four drones hold no defect image. In round r, the server sends the global weights θ^r to every drone. Drone k trains on its n_k questions for 0.25 passes with a new AdamW optimizer and returns θ_k^r. The server averages

$$\theta^{r+1} = \sum_{k=1}^{K} \frac{n_k}{n} \theta_k^r, \qquad n = \sum_k n_k. \tag{1}$$

Ten rounds give 2.5 passes in total, so every run uses the same number of training questions as central training. The learning rate follows a cosine decrease over the rounds. The trained part is the full model of round 2. Each drone keeps 6% of its images out of training. After the last round the drones send the logits and labels of these images, and the server fits the temperatures. No image leaves a drone. The simulation runs the drones one after another on one GPU, and the averaging in Eq. (1) is identical to the FedAvg strategy of Flower [12].

A control runs the same loop with one drone that holds all images, to separate the loop from the federation. We run FedAvg with $K = 3$, 6 and 18 drones, and local only training of the six drones of $K = 6$ without averaging.

- Two starts: the 18 training towers are divided into a server group (13 towers, 2,554 images, 37% defects) and a fleet group (5 towers, 2,553 images, 51% defects), each with all components and conditions. A model is fine-tuned centrally on the server group. Then FedAvg with 5 fleet drones, one per tower, starts either from the published checkpoint or from the server model. This is the server–fleet split of the collaborative direction in [1]: the server trains with the data it may hold, and the fleet improves the model with data it may not share.

Three further sets of runs vary one factor each, with one seed per run:

- Update size (6 tower drones, published checkpoint, 10 rounds): the drone trains and sends the decision head only, the head with LoRA of rank 8 on the seven linear layers of every language layer [20], the head with the last four language layers, the head with all 20 language layers, or the full model. Further runs

send the last-four update in int8 (the change of each tensor in 8 bits with one scale), add FedProx with $\mu = 0.01$ [11], or train the last-four update for 30 rounds with the same local steps per round. After training, the LoRA product is merged into the base weights.

- Few labels (5 fleet drones, one tower each): each drone labels 50, 100 or 200 images drawn from all images of its tower, so about 12% show a defect. Each drone trains one local pass per round for 10 rounds with the last-four update, from the published checkpoint or from the server model. References are the server model alone, the same labels pooled on one server, and each drone alone.
- Image types (6 drones, the 5,107 images, published checkpoint, last-four update): the images are divided at random, by tower, by component (each drone inspects one to three components), or with condition shares drawn from a Dirichlet distribution with $\alpha = 0.3$. In the component split, one drone holds each of corrosion (shackles only), missing caps (glass insulators only) and bird nests (vari-grip clamps only).

3.5. Evaluation

All numbers are on the test towers. For component and condition we report accuracy, balanced accuracy and macro F1, because the majority class covers 90% of the test images. For defect we report AUROC, average precision and expected calibration error (ECE). We tune a threshold on the validation towers that minimizes a risk of 10 per missed defect and 1 per false alarm, and report this risk per 1,000 images. Intervals are 95% bootstrap intervals over 1,000 resamples of the test images. Each configuration has one training seed.

Training sets with a different class mix than the test data shift the probabilities. As a correction that needs no retraining [18], we also report

$$\tilde{p}(c \mid x) \propto p(c \mid x)\, \frac{\pi_{\text{val}}(c)}{\pi_{\text{train}}(c)}, \tag{2}$$

where π are the label shares of the validation towers and of the training images. For comparison with earlier tools on the same benchmark, we also score the 7,057 test items of ElecVQA-Bench. A yes/no item takes $P(\text{defect}) \geq 0.5$, and a multiple-choice item takes the best condition among its options. The reference tools were evaluated on the same items. They are a ResNet-50 [19] classifier fine-tuned on the training towers, SmolVLM-256M tuned with LoRA [20] as in EVLM, and the same SmolVLM without tuning.

4. Results

4.1. Zero-shot

Without fine-tuning, laya-vision does not classify this data. Its condition accuracy is 0.834, below the 0.901 of always answering good. Its macro F1 is 0.180 (95% interval 0.172–0.190), against 0.158 for the majority answer. It calls 480 of the 601 defect images good, and the recall of corrosion, missing cap and bird nest is at most 0.02. The component accuracy is 0.141, below the majority baseline of 0.314. The defect question carries a weak signal (AUROC 0.714), with probabilities that are not calibrated (ECE 0.178). This quantifies the finding of [1] that a pre-trained small VLM gives generic outputs on inspection images.

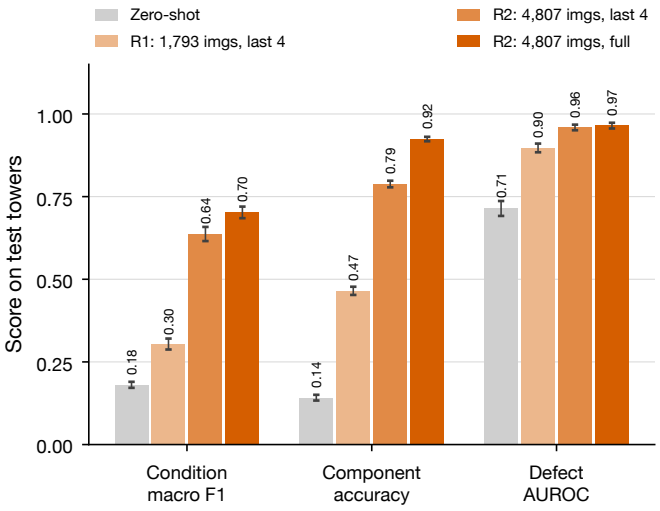


Figure 1: More data and a fully trained backbone lift every metric. Test-tower scores of laya-vision from zero-shot to the round-2 full fine-tune (6,092 images). Whiskers are 95% bootstrap intervals over images; one training seed per model; no class-share correction.

4.2. Fine-tuning

Figure 1 and Table 2 show the progression. Round 1 lifts the defect AUROC to 0.898. Its plain accuracy falls because the model learned a training mix with 79% defects and labels 1,971 of 5,491 good test images as defective. The correction in Eq. (2) raises its condition accuracy from 0.642 to 0.856 and its macro F1 from 0.303 to 0.451. Round 2 adds data, covers all components and has 44% defects. It improves every metric, and the full model is the best: condition accuracy 0.952 (above the majority answer), macro F1 0.703, component accuracy 0.924, defect AUROC 0.965 and ECE 0.058. For this model the correction changes little (condition accuracy 0.957).

Per condition, the full model has F1 0.975 for good, 0.710 for rust, 0.826 for corrosion, 0.762 for missing cap and 0.942 for bird nest. Bird nests are always found. Rust has the most false alarms: 152 good images are called rust, so rust precision is 0.62. The 5 test images of other defects are always missed. Among the components, the spacer is the only one that the model does not recognize (recall 0.06, with 18 training images); all other components have a recall of at least 0.87.

On the ElecVQA-Bench items the full model reaches 0.948, below the fine-tuned ResNet-50 (0.961) and

Table 2: Test-tower results. Images: 6,092 test images. Items: 7,057 ElecVQA-Bench test items. The intervals for laya-vision are in the text and in Fig. 1. Reference tools are scored on the same items.

Model	Cond. macro F1	Comp. acc.	Defect AUROC	Item acc.
Zero-shot	0.180	0.141	0.714	0.830
Round 1, last 4	0.303	0.465	0.898	0.702
Round 2, last 4	0.637	0.789	0.959	0.923
Round 2, full	0.703	0.924	0.965	0.948
ResNet-50, tuned	–	–	–	0.961
SmolVLM + LoRA	–	–	–	0.956
SmolVLM, no tuning	–	–	–	0.830

SmolVLM with LoRA (0.956). Training takes 24 minutes on an RTX 2080 Ti. The test results remain close to the validation results (condition accuracy 0.952 against 0.935, defect AUROC 0.965 against 0.968), so the model does not overfit the training towers.

4.3. Latency

On the RTX 2080 Ti in fp32, one predict call with all four questions takes a median of 72.0 ms per image (p95 98.5 ms), including reading and preprocessing the image. The fine-tuned full model takes 74.0 ms. The condition question alone takes 71.4 ms, so almost all of the time is spent encoding the image, and further questions are nearly free. With 3.45 key frames per 4 s segment, as in EVLM [2], one segment needs 0.25 s.

We have no Jetson measurement. We estimated a factor of 9 between a Quadro GV100 and a Jetson Orin NX from their fp32 throughput and memory bandwidth. The RTX 2080 Ti has a similar fp32 throughput to the GV100, so the same factor gives about 0.67 s per image and 2.2 s per 4 s segment; these are estimates. On a Jetson Orin with the same number of CUDA cores as the Orin NX, [1] measured 2.8 s per image for SmolVLM-256M, and EVLM reports 1.4–6.4 s per segment [2]. If the estimate holds, four answers take about a quarter of the time of one caption. The comparison is approximate: the hardware is not identical, the other systems generate text while laya-vision returns labels, and EVLM processes video segments while we process single crops.

4.4. The decision model as a semantic trigger

The agenda of [1] calls for hierarchical pipelines in which an inexpensive stage decides which frames reach the VLM, and proposes a detector for this stage. A detector localizes components but does not judge their condition. The decision model does judge the condition, so it can serve as the trigger itself. We use the defect threshold tuned on the validation towers (0.057). On the test towers it passes 931 of 6,092 images (15.3%) to a following stage and finds 532 of the 601 defects (recall 0.885), with a false-alarm rate of 0.073 among good images and a risk of 179 per 1,000 images, against 987 for always answering good. These rates are computed from the per-image predictions; the cascade itself is not deployed.

4.5. Calibration and confidence

A mission planner can act on a probability only if the probability is reliable. After fine-tuning, the defect probability has an ECE of 0.058 (0.178 zero-shot) and an average precision of 0.862. The judgeable question, which asks the model whether it can judge the image, does not work as a confidence signal: its AUROC for a correct condition answer is 0.53 zero-shot and 0.31 after fine-tuning. Confidence for a decision model should therefore come from its calibrated answer probabilities, not from a self-assessment question.

4.6. Federated fine-tuning across drones

The control with one drone matches central training: macro F1 0.697 against 0.703 (Table 3). The intervals overlap for all three metrics, so the training loop does not cause the differences below. With the same data spread over more drones, FedAvg loses accuracy (Fig. 2b, c). From 1 to 18 drones, the condition macro F1 falls from 0.697 to 0.474 and the component accuracy from 0.933 to 0.700. The defect AUROC degrades less, from 0.959 to 0.933, so the trigger of Section 4.4 is the most robust output under federation. The 6- and 18-drone runs are still improving after ten rounds (Fig. 2a), while the control has levelled off; Section 4.8 tests more rounds.

Federation still helps the individual drone. With six drones, FedAvg beats the average drone that trains alone on every metric (macro F1 0.541 against 0.371, defect AUROC 0.939 against 0.881) and the best single drone in defect AUROC (0.939 against 0.916); the best drone's macro F1 (0.525) lies inside the FedAvg interval.

4.7. A fine-tuned start for the fleet

The lower part of Table 3 compares the two starts on the same fleet. From the published checkpoint, fleet FedAvg reaches a macro F1 of 0.513, a component accuracy of 0.593 and a defect AUROC of 0.932. From the server model it reaches 0.636, 0.912 and 0.953, and the intervals of the two starts do not overlap. The fleet also improves the server model, from 0.558 to 0.636 macro F1 and from 0.903 to 0.953 AUROC, while its images stay on the drones. Its item accuracy (0.937) is close to that of central training on all 18 towers (0.948).

4.8. What each drone sends

The trained part of the model is the part each drone sends. Table 4 gives the size of each choice, counted from the checkpoint, with its result after 10 FedAvg rounds over six tower drones. The vision encoder (86.4M parameters) is frozen in every choice, and token embeddings are a quarter of the full update.

From the published checkpoint, the small updates learn little in 10 rounds (Fig. 3). The head alone reaches a macro F1 of 0.243, LoRA 0.247 and the last four layers 0.281, close to the zero-shot model, while the full update

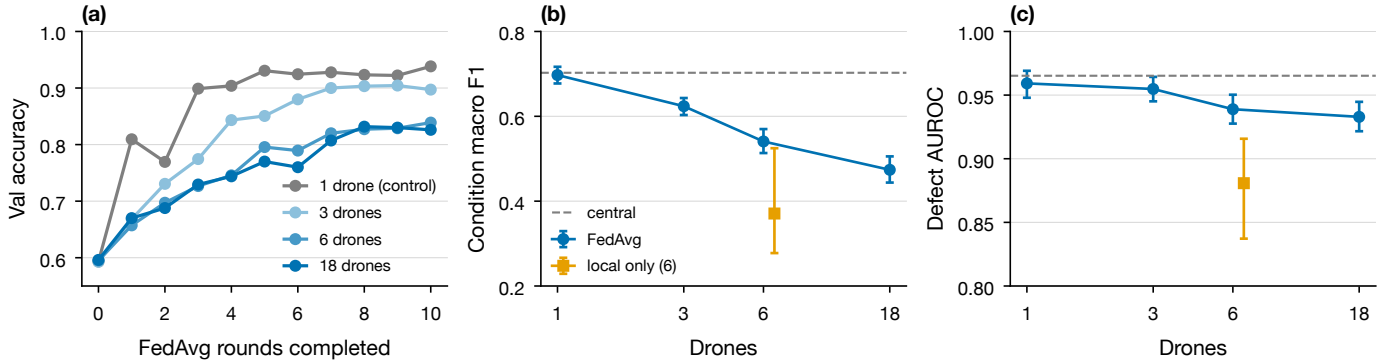


Figure 2: Splitting the same data over more drones slows FedAvg. (a) Validation accuracy on 600 images after each round. (b, c) Test-tower condition macro F1 and defect AUROC against the number of drones. FedAvg whiskers are 95% bootstrap intervals. The local-only marker is the mean of six drone models, and its whisker spans their minimum to maximum. The dashed line is central training. Every run uses the same total training (2.5 passes), with one seed per run.

Table 3: Federated runs on the test towers, with 95% bootstrap intervals. Local only: mean (minimum–maximum) of the six drone models. Uplink: full-model update in fp16 (229.5 MB) × drones × 10 rounds, calculated, not measured.

Run	Cond. macro F1 (95% interval)	Comp. acc.	Defect AUROC	Uplink
Central (round 2, full)	0.703 (0.685–0.720)	0.924	0.965	–
Control, 1 drone	0.697 (0.677–0.717)	0.933	0.959	–
FedAvg, 3 drones	0.624 (0.603–0.643)	0.874	0.955	6.9 GB
FedAvg, 6 drones	0.541 (0.513–0.570)	0.761	0.939	13.8 GB
FedAvg, 18 drones	0.474 (0.444–0.505)	0.700	0.933	41.3 GB
Local only, 6 drones	0.371 (0.278–0.525)	0.621	0.881	–
Server towers, central	0.558 (0.532–0.584)	0.894	0.903	–
Fleet FedAvg, base start	0.513 (0.484–0.540)	0.593	0.932	11.5 GB
Fleet FedAvg, fine-tuned start	0.636 (0.612–0.657)	0.912	0.953	11.5 GB

Table 4: Trained and sent part of laya-vision, the size of one drone update (2 bytes per parameter in fp16), and the test result after FedAvg with six tower drones from the published checkpoint (10 rounds, one seed). Central training of the full model: 0.703 and 0.965.

Trained and sent	Params	Update	Macro F1	AUROC
Decision head	8.46M	16.9 MB	0.243	0.792
Head + LoRA rank 8	10.09M	20.2 MB	0.247	0.828
Head + last 4	22.62M	45.2 MB	0.281	0.838
same, int8	22.62M	22.6 MB	0.278	0.838
same, FedProx	22.62M	45.2 MB	0.275	0.834
same, 30 rounds	22.62M	45.2 MB	0.488	0.945
Head + 20 layers	79.27M	158.5 MB	0.488	0.931
Full (no vision enc.)	114.73M	229.5 MB	0.541	0.939
same, FedProx	114.73M	229.5 MB	0.526	0.940

reaches 0.541. Trained centrally, the same last-four update reaches 0.637 (Table 2), so the small update itself is not the problem. With 30 rounds, the last-four update reaches 0.488 and a defect AUROC of 0.945, and its validation accuracy is still rising (0.66 after 10 rounds, 0.81 after 30). Our reading, supported but not proven, is that averaging six small changes per round moves the shared model slowly, and an update with more parameters moves faster per step.

int8 gives the same result as fp16 (macro F1 0.278 against 0.281, AUROC 0.838 for both), so it halves the uplink for free; FedProx with $\mu = 0.01$ changes nothing for either update.

A full-model run needs 6.9–41.3 GB of uplink and the int8 last-four run 1.4 GB, against about 0.18 GB to upload all 5,107 training images once. Federation therefore keeps images on the drones rather than saving bandwidth, and updates would be sent from the ground station after a mission.

4.9. Few labels per drone

A real fleet labels few images, mostly good ones. From the published checkpoint, 50 to 200 natural labels per drone give almost nothing: the macro F1 stays at 0.159–0.207, near the 0.158 of always answering good (Fig. 4). From the server model, 100 and 200 labels per drone raise the defect AUROC from 0.903 to 0.930 and 0.933, while the macro F1 changes little (0.558 to 0.582 and 0.575, overlapping intervals). Fifty labels per drone hurt: the AUROC drops to 0.836 and the defect risk rises from 291 to 331 per 1,000 images. Pooling the same 200 labels per drone on one server is better (AUROC 0.958 against 0.933, risk 181 against 280), and FedAvg beats the average drone alone (AUROC 0.933 against 0.902).

4.10. Drones with different image types

The Jensen–Shannon divergence between the condition shares of a drone and of the fleet is at most 0.002 for the

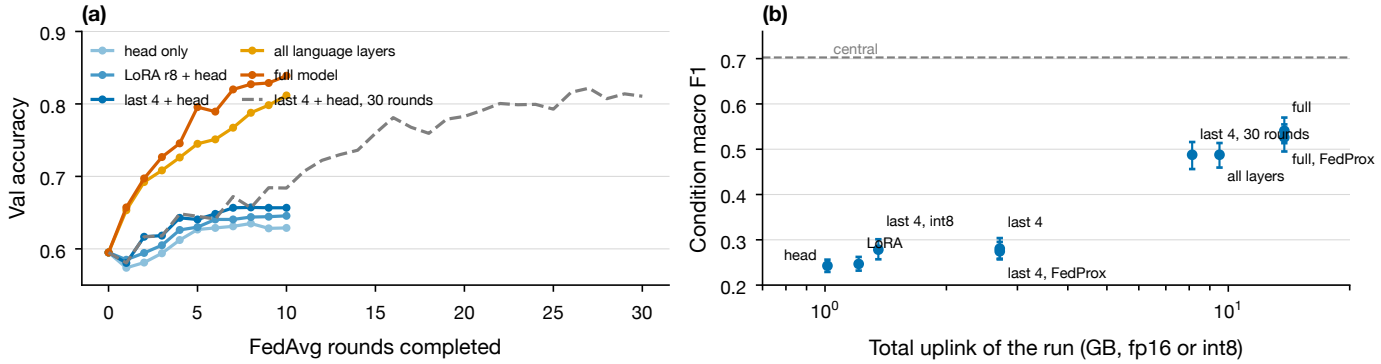


Figure 3: From the published checkpoint, small updates learn slowly under FedAvg. Six tower drones, 10 rounds unless stated. (a) Validation accuracy on 600 images after each round. (b) Test-tower condition macro F1 against the total uplink of the run (update size × 6 drones × rounds, calculated); whiskers are 95% bootstrap intervals; the dashed line is central training of the full model. One seed per run.

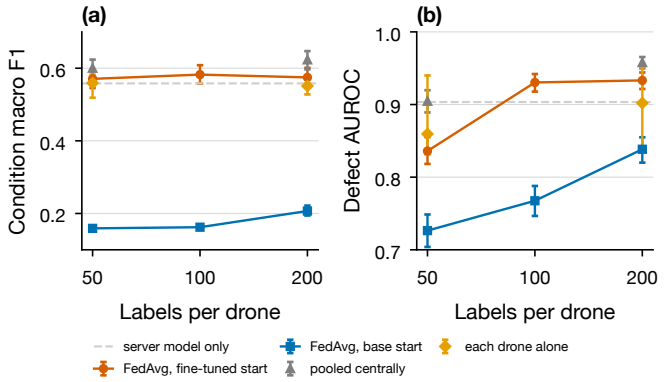


Figure 4: With few labels, only the fine-tuned start helps. Five fleet drones, labels drawn with the natural defect share (about 12%), last-four update, 10 rounds. Whiskers: 95% bootstrap intervals; for drones alone, the range of the five drone models. The dashed line is the server model without fleet labels. One seed per run; runs with 25 labels are not included.

random split, 0.009–0.070 by tower, 0.128–0.333 by component and 0.090–0.665 for the Dirichlet split. With the last-four update from the published checkpoint, all splits give the same result within their intervals: macro F1 0.254 (random), 0.281 (tower), 0.287 (component), 0.285 (component with FedProx) and 0.291 (Dirichlet).

Table 5 shows why the split does not matter here. The drone that holds a defect type learns it well when it trains alone, but FedAvg with the last-four update loses all three types, and it loses them just as much with the random split, where every drone sees every type. The cause is therefore the slow small update, not the differences between drones; with this update the experiment cannot measure the effect of the split. Training alone has its own cost: such a drone recognizes the other components poorly (component accuracy 0.16 on average, against 0.33 for FedAvg).

Table 5: Test recall of the three defect types that only one drone holds in the component split. Drone alone: the drone that holds the type, trained without averaging. FedAvg: last-four update from the published checkpoint, six drones, 10 rounds; the last column is the full-model FedAvg of Table 3.

Defect type	Drone alone	FedAvg component	FedAvg random	FedAvg full tower
Corrosion (shackles)	0.86	0.07	0.01	0.76
Missing cap (insulators)	0.79	0.02	0.03	0.37
Bird nest (vari-grip)	0.94	0.12	0.00	0.67

5. Discussion

What the decision model adds to semantic edge inspection. The agenda of [1] frames semantic inspection as description. Our results show that a large part of the inspection decision does not need description. With the same SmolVLM backbone family, a decision model answers component, defect and condition questions with measured accuracy close to a fine-tuned describing model, from one image encoding. It needs no synthetic captions and gives probabilities that can drive a trigger; the describing VLM remains useful for reports and for defects outside the option set.

What federation adds. The collaborative direction of [1] becomes testable once the onboard model is a classifier with a fixed output set. FedAvg across tower-wise drones beats training alone, and a server-fine-tuned start brings the fleet close to central training. The costs are accuracy as the data is split further under a fixed budget, and an uplink of several gigabytes per run for the full model. Small updates cut the uplink and int8 is free, but from the published checkpoint they need many more rounds. The recipe that works throughout is to fine-tune centrally on the data the operator may hold and let the fleet continue; with few labels, federation pays only where the labels cannot be pooled.

Limitations. Each configuration has one seed. The drones are simulated on one GPU, so uplink is calculated, not

measured. The Jetson latency is an estimate, and energy is not measured. The fixed compute budget gives FedAvg with many drones fewer effective steps. The data covers one benchmark and one asset type, with few images in some classes. The trigger is evaluated offline. The update-size and image-type runs start from the published checkpoint, where small updates learn slowly, and the runs with 25 labels per drone are not included.

Future work. Small updates and image-type splits from a fine-tuned start, longer FedAvg runs, more seeds, a Jetson measurement of latency and energy, a deployed cascade, and a second asset type.

6. Conclusion

A 201M-parameter vision-language decision model fails zero-shot on power-line component images, but after fine-tuning on folder labels it reaches 0.948 item accuracy at 74 ms per image for four questions, with calibrated defect probabilities that can act as a trigger. FedAvg across tower-wise drones beats training alone, and a start fine-tuned on separate server towers brings the fleet close to central training. From the published checkpoint, small updates learn little in ten rounds; int8 halves an update at no measurable cost and FedProx does not help. With 100–200 natural labels per drone, fleet FedAvg from the server model improves defect ranking, but less than pooling the same labels.

Data and code availability

The code, the per-image predictions and the evaluation files are in the project repository, branch vision-fl (to be made public on acceptance). ElecVQA-Bench and InsPLAD are public [16, 17]. The laya-vision weights are published under CC BY-NC-SA 4.0 [6].

Declaration of competing interest

The author declares no competing interests.

CRediT authorship contribution statement

Mahdi Khashan: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft, Writing – review & editing.

Declaration of generative AI use

During the preparation of this work the author used an AI assistant to write code, analyse results and draft text. The author reviewed and edited the content and takes full responsibility for the publication.

Acknowledgments

Experiments in this work were run and tracked with OpenResearch [21].

References

[1] R. Farahani, N. Hossein Motlagh, Z. Azimi, C. Timmerer, L. Carnevale, S. Tarkoma, S. Dustdar, From pixels to semantics: Edge AI for UAV-based critical infrastructure inspection, arXiv:2609.18448 (2026).

[2] R. Farahani, Z. Azimi, I. Murturi, A. Goknil, S. Sen, C. Timmerer, S. Dustdar, EVLM: Intent-driven edge vision language model for UAV-based power line inspection, in: 2026 IEEE International Conference on Edge Computing and Communications (EDGE), 2026. https://doi.org/10.1109/EDGE72783.2026.00014.

[3] Z. Azimi, R. Farahani, S. Dustdar, C. Timmerer, Cloud, edge, or split? Profiling onboard and split vision-language model deployment for drone AI, arXiv:2609.25415 (2026).

[4] A. Marafioti, et al., SmolVLM: Redefining small and efficient multimodal models, arXiv:2504.05299 (2025).

[5] Convai Innovations, Laya, software (2026). https://github.com/NandhaKishorM/laya.

[6] Laya Vision, software, commit 9e1e241 (2026). https://github.com/r33drichards/laya-vision; weights https://huggingface.co/thaitea/laya-vision.

[7] Sabanovic, Maliakel, Brandic, INAR-VL, arXiv:2605.18853 (2026).

[8] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, On calibration of modern neural networks, in: Proceedings of the 34th International Conference on Machine Learning (ICML), 2017.

[9] B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), 2017.

[10] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, V. Chandra, Federated learning with non-IID data, arXiv:1806.00582 (2018).

[11] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, V. Smith, Federated optimization in heterogeneous networks, in: Proceedings of Machine Learning and Systems (MLSys), 2020.

[12] D. J. Beutel, et al., Flower: A friendly federated learning research framework, arXiv:2007.14390 (2020).

[13] Askin, et al., Federate the router, arXiv:2601.22318 (2026).

[14] Solat, Lee, Seif, Niyato, Poor, FedHLM, IEEE Internet of Things Journal (2025). https://doi.org/10.1109/jiot.2025.3618095.

[15] From open-vocabulary detection to structural assessment: An agentic federated approach for guardrail monitoring (2026). https://www.sciencedirect.com/science/article/pii/S2772503026000459.

[16] ElecVQA-Bench, dataset and code repository (2026). https://github.com/GhostDino/elecvqa-bench.

[17] A. L. B. Vieira e Silva, et al., InsPLAD: A dataset and benchmark for power line asset inspection in UAV images, International Journal of Remote Sensing (2023). arXiv:2311.01619.

[18] M. Saerens, P. Latinne, C. Decaestecker, Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure, Neural Computation 14 (1) (2002) 21–41.

[19] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[20] E. J. Hu, et al., LoRA: Low-rank adaptation of large language models, in: International Conference on Learning Representations (ICLR), 2022.

[21] alphaXiv, OpenResearch: The local-first harness and workspace for research agents (2026). https://github.com/alphaXiv/OpenResearch.